22 ROLE-BASED AUTHORIZATION IN NESTJS GUARDS (RBAC)

Role-Based Authorization (RBAC = Role-Based Access Control) answers the AUTHORIZATION half of theory/18-authentication-vs-authorization.txt: given a request we already know WHO is calling (authentication already ran), is THIS role allowed to hit THIS route?

```
Client Request
     |
AuthGuard        <-- AUTHENTICATION: verifies identity,
     |                   attaches request.user (or throws 401)
RolesGuard        <-- AUTHORIZATION: reads request.user.role +
     |                   the route's required roles (throws 403)
Route Handler runs
```

The whole point of RBAC is to avoid writing a bespoke guard per permission rule. Instead you write ONE generic RolesGuard, and tag each route with WHICH roles it needs using a custom decorator + metadata.

STEP 1 — GIVE THE AUTHENTICATED USER A ROLE

This project's real guard, src/guards/auth/auth.guard.ts, currently only proves possession of a shared API key — it never identifies WHO is calling, so there is no role to check against (see theory/18, the "crude authentication" section). RBAC needs request.user.role to exist, so the first change is to have the guard ATTACH a user:

```ts
// src/guards/auth/auth.guard.ts (conceptual extension)
if (key !== VALID_API_KEY) {
  throw new UnauthorizedException('Invalid or missing API key');
}
request['user'] = { id: 1, role: 'admin' }; // normally decoded from a JWT
return true;
```

In a real app this comes from decoding a JWT (see the AuthGuard example in theory/18-authentication-vs-authorization.txt) or looking up a session — the key requirement for RBAC is just: SOME earlier guard must populate request.user.role before RolesGuard runs.

STEP 2 — A CUSTOM DECORATOR TO DECLARE REQUIRED ROLES

@SetMetadata attaches arbitrary metadata to a route handler, readable later via Reflector (see theory/06-decorators.txt for decorators in general). Wrapping it in a small function gives a clean, typed API:

```ts
// src/common/decorators/roles.decorator.ts
import { SetMetadata } from '@nestjs/common';

export const ROLES_KEY = 'roles';
export const Roles = (...roles: string[]) => SetMetadata(ROLES_KEY, roles);

// usage on a route
@Roles('admin')
```

```
@Delete(':id')
remove(...) { ... }
```

@Roles('admin') does NOT enforce anything by itself — it just stamps the string array ['admin'] onto the remove handler's metadata. Nothing reads it yet; that's RolesGuard's job.

STEP 3 — RolesGuard: THE ONE GUARD THAT ENFORCES EVERY RULE

```
// src/guards/roles/roles.guard.ts
import {
  CanActivate,
  ExecutionContext,
  ForbiddenException,
  Injectable,
} from '@nestjs/common';
import { Reflector } from '@nestjs/core';
import { ROLES_KEY } from '../../common/decorators/roles.decorator';

@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private readonly reflector: Reflector) {}

  canActivate(context: ExecutionContext): boolean {
    const requiredRoles = this.reflector.get<string[]>(
      ROLES_KEY,
      context.getHandler(),
    );

    if (!requiredRoles || requiredRoles.length === 0) {
      return true; // route has no @Roles() -> no restriction
    }

    const { user } = context.switchToHttp().getRequest();

    if (!user) {
      // RolesGuard ran without an authenticated user ahead of it —
      // a wiring mistake (see STEP 4 ordering), not a role problem
      throw new ForbiddenException('No authenticated user on request');
    }

    if (!requiredRoles.includes(user.role)) {
      throw new ForbiddenException(
        `Requires role(s): ${requiredRoles.join(', ')}`,
      ); // 403 — authenticated fine, just not permitted
    }

    return true;
  }
}

reflector.get(KEY, context.getHandler())   Reads metadata set by
                                            @SetMetadata on THIS
                                            specific route handler.
                                            context.getHandler() is
                                            the exact controller
                                            method being called
                                            (as opposed to
```

```
                                    context.getClass(), which
                                    would read
                                    controller-level metadata
                                    — see theory/17-guards.txt
                                    for the ExecutionContext
                                    basics).
```

Constructor-injecting Reflector is why RolesGuard should be applied via DI (@UseGuards(RolesGuard) lets Nest construct it through the container) rather than `new RolesGuard()` by hand — same caveat as any guard/pipe that needs a dependency (see theory/11-builtin-vs-custom-pipes.txt).

STEP 4 — WIRING IT INTO task.controller.ts (ORDER MATTERS)

```ts
import { Roles } from '../common/decorators/roles.decorator';
import { RolesGuard } from '../guards/roles/roles.guard';

@Roles('admin')
@UseGuards(AuthGuard, RolesGuard)   // AuthGuard FIRST, RolesGuard SECOND
@Delete(':id')
remove(@Param('id', ParsePositiveIntPipe) id: number): Task {
  return this.taskService.remove(id);
}
```

Guards passed to @UseGuards() run LEFT TO RIGHT, and Nest calls each in sequence — the next guard only runs if the previous one returned true. AuthGuard must come first because RolesGuard reads request.user, which only exists once AuthGuard has attached it. Swapping the order (or applying RolesGuard alone) means request.user is undefined and every role check fails, regardless of who's actually calling.

Three outcomes for DELETE /task/1 with this wiring:

- Missing/wrong x-api-key → AuthGuard throws → 401
- Valid key, user.role != 'admin' → RolesGuard throws → 403
- Valid key, user.role == 'admin' → both pass, handler runs → 200

APPLYING RolesGuard GLOBALLY (SO IT'S NEVER FORGOTTEN)

Since RolesGuard is a NO-OP on any route without @Roles(...) (it returns true immediately), it's safe — and common — to register it once for the whole app via APP_GUARD (see theory/17-guards.txt option 4), rather than repeating @UseGuards(RolesGuard) on every protected route:

```ts
// app.module.ts
import { APP_GUARD } from '@nestjs/core';

@Module({
  providers: [
    { provide: APP_GUARD, useClass: AuthGuard },   // authenticate everything
    { provide: APP_GUARD, useClass: RolesGuard },  // then check roles, per-route
  ],
})
export class AppModule {}
```

With this in place, task.controller.ts only needs the declarative @Roles('admin') tag — no @UseGuards() at all — and any route WITHOUT @Roles() stays open to any authenticated user, since RolesGuard's early `if (!requiredRoles) return true;` lets it through.

MULTIPLE ROLES AND "ANY OF" vs "ALL OF"

@Roles('admin', 'editor') passes an array; the guard above uses requiredRoles.includes(user.role) — this is an "ANY OF" check (user needs ONE matching role). An "ALL OF" check (user needs EVERY listed role) is rarer for role systems but would swap to:

```
const hasAll = requiredRoles.every((r) => user.roles.includes(r));
```

— note this also assumes user.roles is an ARRAY (multi-role users), not the single user.role string used above. Pick the shape (single role vs role array per user) based on whether your app ever needs one user to hold more than one role at once.

⚠ COMMON MISTAKES

Checking role in the SERVICE instead of a guard Pushes an authorization concern past the point where Nest can cleanly reject the request with a 403 before any business logic runs — duplicates the check into every service method that needs it. Keep authorization at the guard layer (see the Guard/Pipe/Interceptor split in theory/17-guards.txt).

RolesGuard with no preceding AuthGuard requiredRoles.includes(user.role) throws on `user` being undefined, or silently passes if you forget the undefined check — always guard against a missing request.user explicitly (STEP 3 above throws ForbiddenException rather than crashing).

Returning 401 for a role mismatch A valid, authenticated user who lacks the right role is a 403 case, not 401 — see theory/18-authentication-vs-authorization.txt for why conflating these misleads API clients.

```
Reading metadata with context.getClass() when you meant getHandler()
    getClass() reads CONTROLLER-level @SetMetadata, getHandler()
    reads the specific ROUTE METHOD's metadata. Using the wrong one
    means @Roles() on one method leaks into (or fails to affect)
    others. Nest's own official RolesGuard example actually checks
    BOTH, merging controller-level and method-level roles — worth
    doing if you want a controller-wide default that individual
    routes can override.
```

★ RULE OF THUMB

Authenticate once (populate request.user), then authorize with ONE reusable RolesGuard driven by a small @Roles(...) decorator per route — never hardcode a role string inside a guard, and never duplicate the role check inside a service. Order guards so identity is established before permissions are checked: AuthGuard, then RolesGuard, always in that sequence.

See also: **theory/17-guards.txt**, **theory/18-authentication-vs-authorization.txt**, **theory/06-decorators.txt**, **theory/07-dependency-injection.txt** </content>